

Decroly SANTANDER

UD3

API REST CON SPRING BOOT

MP_0613

Desarrollo Web en Entorno Servidor

3.1 Tu primer proyecto Spring Boot

Qué vas a aprender en este tema

Con este tema empieza el bloque de Spring Boot. Vas a crear desde cero el proyecto **tienda-decroly**, que irá creciendo tema a tema hasta convertirse en el backend completo de una tienda online. Antes de escribir código necesitas entender tres cosas: **cómo se organiza** una aplicación de servidor moderna, **qué es Spring Boot** y qué hace por ti, y **qué ocurre** desde que el navegador pide una dirección hasta que tu código responde.

Del monolito a la arquitectura por capas

Cómo programabas en el PHP

Con PHP programabas todo en un mismo fichero que hacía de todo: generaba el HTML, leía los datos del formulario, hacía los cálculos y, a veces, consultaba la base de datos. Por ejemplo, una página `productos.php` de una tienda podría contener:

productos.php (estilo Bloque 1)

```
<h1>Nuestros productos</h1>                    <!-- presentacion -->
<?php
  $conexion = new mysqli("localhost", ...);     // acceso a datos
  $res = $conexion->query("SELECT * FROM producto");
  while ($p = $res->fetch_assoc()) {
    $precio = $p["precio"] * 1.21;               // logica de negocio
    echo "<p>" . $p["nombre"] . ": " . $precio . " EUR</p>";
  }
?>
```

Funciona, pero tiene problemas en cuanto la aplicación crece:

- **No se puede reutilizar.** Si la tienda quiere una app móvil, el cálculo del IVA y la consulta están atrapados dentro de un fichero que genera HTML. Habría que copiarlos.
- **Un cambio afecta a todo.** Si el IVA cambia, o cambia la base de datos, hay que buscar y modificar todas las páginas que lo usan.
- **Es difícil trabajar en equipo.** El diseñador y el programador tocan el mismo fichero.
- **Es difícil de probar.** No se puede comprobar el cálculo del precio sin ejecutar la página entera y mirar el HTML.

Una aplicación construida así, como una sola pieza en la que todo está mezclado, se llama **monolito**.

La separación frontend y backend

Con la llegada de los móviles, las tablets y las aplicaciones JavaScript, un mismo servidor tiene que dar servicio a muchos clientes distintos. Por eso se separó el **frontend** (lo que ve el usuario:

una web, una app) del **backend** (el servidor que guarda los datos y aplica las reglas). El backend ya no genera páginas: ofrece **datos y operaciones** que cualquier frontend puede usar, normalmente mediante **REST**, un estilo de comunicación sobre HTTP que estudiarás a fondo más adelante.

Algunas empresas van más allá y dividen el backend en varias aplicaciones pequeñas e independientes, llamadas **microservicios**: una para el catálogo, otra para los pedidos, otra para los usuarios.

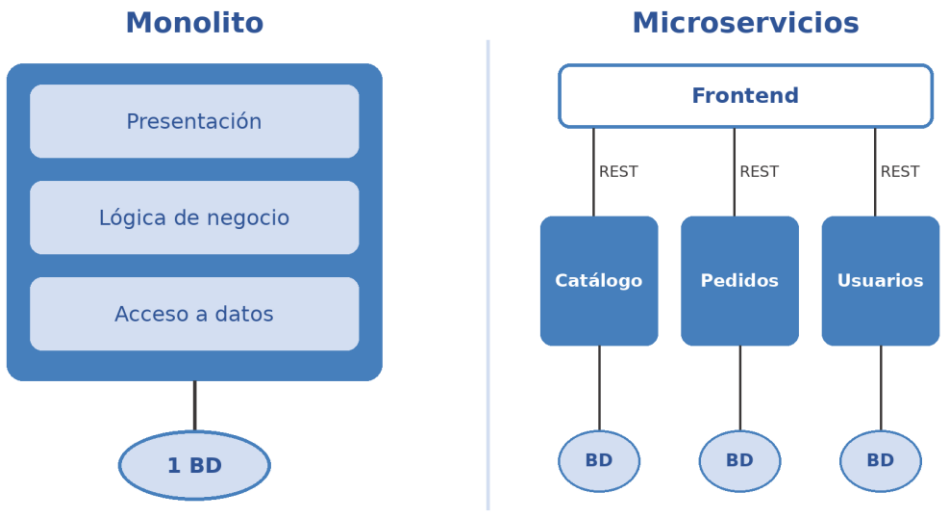


Figura 1. Un monolito frente a un sistema dividido en microservicios que se comunican por REST.

	Monolito	Microservicios
Despliegue	Una sola aplicación	Varias aplicaciones independientes
Un fallo en una parte…	Puede tirar toda la aplicación	Solo afecta a ese servicio
Escalado	Se escala todo a la vez	Se dan más recursos solo al servicio más usado
Equipos	Todos tocan el mismo código	Cada equipo se centra en su servicio
Complejidad	Sencillo de arrancar y desplegar	Más difícil de coordinar y desplegar

La arquitectura por capas

No hace falta llegar a los microservicios para organizar bien el código. La solución más extendida es dividir la aplicación en **capas**, cada una con **una sola responsabilidad**. Cada capa solo habla con la que tiene justo debajo.

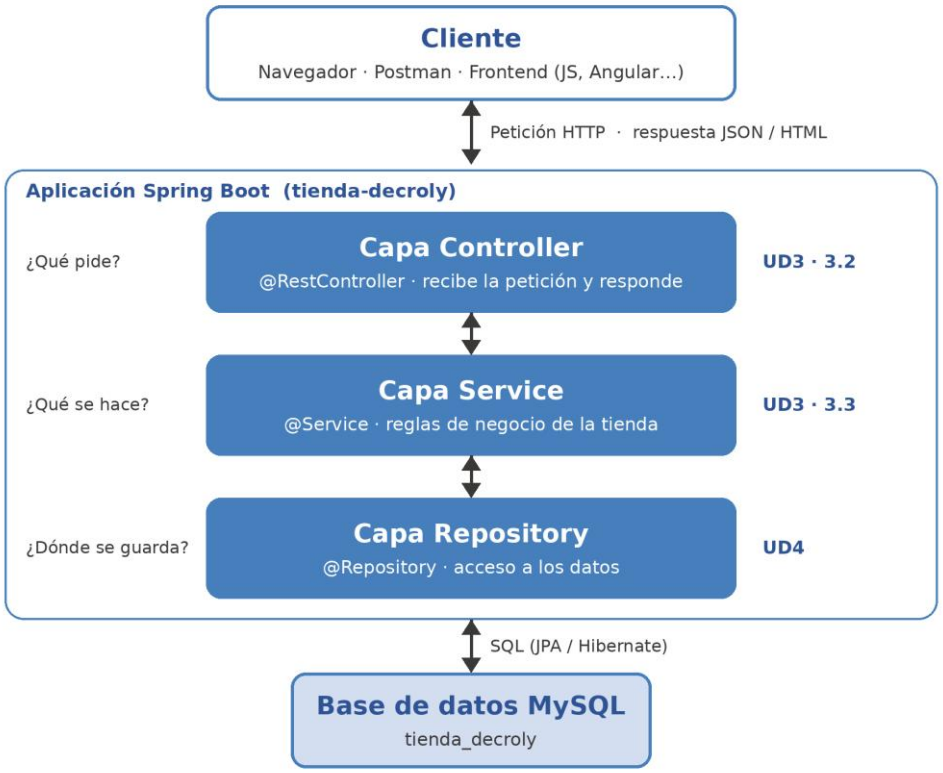


Figura 2. Arquitectura por capas de tienda-decroly y tema en el que trabajarás cada una.

Capa	Pregunta que responde	Responsabilidad	Ejemplo en la tienda
Controller	¿Qué me piden?	Recibe la petición HTTP, extrae sus datos, llama al servicio y devuelve la respuesta. No toma decisiones de negocio.	Atender `GET /api/productos` y devolver la lista.
Service	¿Qué hay que hacer?	Aplica las reglas de negocio. No sabe nada de HTTP ni de SQL.	Calcular el precio con IVA. Comprobar que hay stock antes de aceptar un pedido.
Repository	¿Dónde se guarda?	Guarda y recupera los datos. No sabe nada de las reglas del negocio.	Buscar en MySQL los productos de una familia.

Si reescribimos el ejemplo de `productos.php` por capas, el cálculo del IVA vive en un solo sitio, el **servicio**. La web, la app móvil y cualquier otro cliente lo reutilizan sin copiarlo. Si cambia el IVA, se cambia una línea. Si se cambia MySQL por otra base de datos, solo cambia el repositorio. Y el cálculo se puede probar por separado, sin servidor ni HTML.

> **Nuestra tienda: un único servicio, bien organizado por dentro**
>
> No vamos a construir microservicios: la tienda será **una sola aplicación Spring Boot**, pero dividida en capas. Más adelante comprobarás la ventaja de esta separación: mostrarás los mismos productos como datos JSON y como página web sin tocar ni una línea de la lógica de negocio.

2. Qué es Spring y qué es Spring Boot

Spring Framework

Spring es el framework más utilizado para desarrollar aplicaciones empresariales en Java. Nació en 2003 y hoy es una familia de proyectos. En este bloque usarás varios de ellos:

Proyecto	Para qué sirve
Spring MVC (Spring Web)	Aplicaciones web y servicios REST
Spring Data JPA	Acceso a bases de datos
Spring Security	Autenticación, roles y protección de la aplicación
Spring Boot	Configura y arranca todo lo anterior automáticamente

La idea central: el contenedor de Spring

En un programa Java normal, **tú** creas los objetos con `new` y los conectas entre sí. En una aplicación grande eso significa cientos de `new` en el orden correcto. Spring invierte esa responsabilidad: tú **marcas tus clases con anotaciones** y, al arrancar, un componente de Spring llamado **contenedor** crea un objeto de cada una y los conecta donde hacen falta. Los objetos que crea y gestiona el contenedor se llaman **beans**.

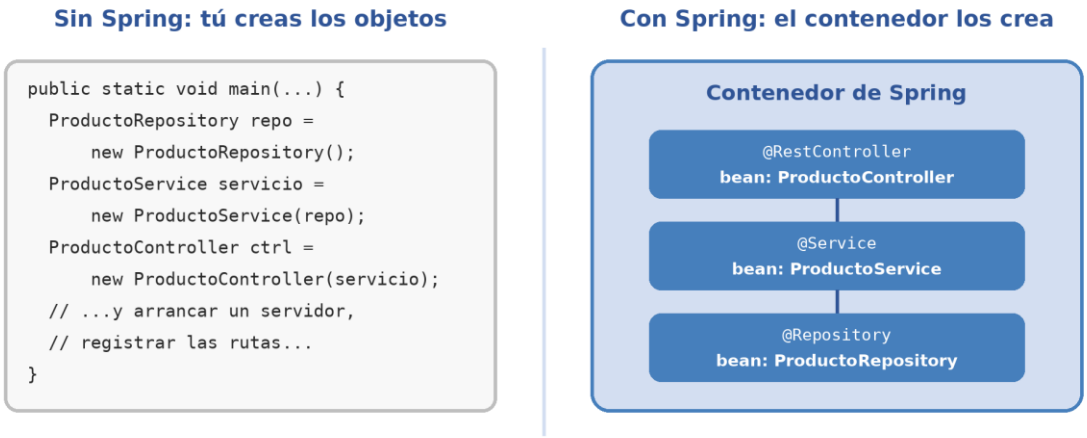


Figura 3. Sin Spring creas y conectas tú los objetos; con Spring lo hace el contenedor.

A esta idea se la conoce como **inversión de control**: ya no es tu código quien controla la creación de los objetos, sino el framework. Cómo se conectan unos beans con otros (la *inyección de dependencias*) lo estudiarás más adelante, al crear la capa de servicios.

Qué añade Spring Boot

Spring es muy potente, pero configurarlo a mano era largo: había que declarar cada componente, elegir versiones compatibles de decenas de librerías, instalar un servidor Tomcat aparte y desplegar la aplicación en él. **Spring Boot** elimina ese trabajo con cuatro ideas:

Idea	Qué significa
Starters	Paquetes de dependencias ya probadas juntas. Con una sola dependencia (`spring-boot-starter-web`) tienes todo lo necesario para una aplicación web.
Autoconfiguración	Spring Boot mira qué librerías tienes y se configura solo. Si ve Spring Web, prepara un servidor web; si ve un driver de MySQL, prepara la conexión.
Servidor embebido	Tomcat viaja **dentro** de tu aplicación. No hay que instalarlo ni desplegar nada en él.
Jar ejecutable	Toda la aplicación, con sus librerías y su servidor, se empaqueta en un fichero `.jar` que se ejecuta con `java -jar`.

A esto se le llama **convención sobre configuración**: Spring Boot aplica valores razonables por defecto y tú solo configuras lo que quieres cambiar.

Spring Boot	
Servidor	Tomcat embebido en la aplicación

	Spring Boot
Puesta en marcha	Ejecutar la clase principal o el .jar
Librerías	Declaradas en `pom.xml`; Maven las descarga
Organización	Capas separadas: controller, service, repository

Las anotaciones de Java

Spring se apoya continuamente en las **anotaciones**, esas palabras que empiezan por `@`. Ya conoces una: `@Override`. Una anotación es una **etiqueta** que se coloca sobre una clase, un método, un atributo o un parámetro para añadir información (metadatos). La anotación **no ejecuta nada por sí misma**: es otro programa el que la lee y actúa en consecuencia. Con `@Override`, quien la lee es el compilador; con las anotaciones de Spring, es el contenedor al arrancar.

Anotaciones sobre una clase y un método

```
@RestController              // en la clase: "es un controlador web"
public class HolaMundoRestController {

    @GetMapping("/saludo")    // en el metodo: "atiende GET /saludo"
    public String saludo() {
        return "Hola";
    }
}
```

Estas son las anotaciones más importantes que usarás a lo largo del bloque. No hace falta memorizarlas ahora:

Anotación	Se coloca en	Significado
`@SpringBootApplication`	Clase principal	Punto de arranque de la aplicación
`@RestController`	Clase	Controlador que devuelve datos
`@GetMapping`, `@PostMapping`…	Método	Asocia el método a una ruta y un método HTTP
`@Service`	Clase	Clase de la capa de servicios
`@Repository`	Clase o interfaz	Clase de la capa de datos
`@Entity`	Clase	Clase que se guarda en una tabla

3. Cómo se estructura un proyecto Spring Boot

Antes de crear el proyecto, vamos a ver cómo es por dentro. Así, cuando lo generes, sabrás qué es cada cosa.

Maven: el gestor del proyecto

Un proyecto Spring Boot es, ante todo, un **proyecto Maven**. **Maven** es la herramienta que gestiona los proyectos Java y hace tres trabajos:

- **Gestiona las dependencias.** Tú declaras qué librerías necesitas; Maven se encarga de descargalas del **repositorio central** de Internet y las guarda en tu ordenador, en la carpeta `.m2` de tu usuario. Si una librería necesita otras, Maven las descarga también (son las *dependencias transitivas*).
- **Construye el proyecto.** Compila, ejecuta las pruebas y empaqueta, siempre con los mismos pasos.
- **Impone una estructura estándar** de carpetas, de modo que cualquier programador Java se orienta en cualquier proyecto.

Es el equivalente a `npm` en JavaScript. Toda la configuración está en un único fichero: `pom.xml`.

La estructura de carpetas

Estructura de tienda-decroly recién creado

```
tienda-decroly/
|-- .mvn/wrapper/                <- Maven Wrapper
|-- src/
|   |-- main/
|   |   |-- java/es/decroly/tienda/
|   |   |   `-- TiendaDecrolyApplication.java  <- clase principal
|   |   `-- resources/
|   |       |-- static/          <- CSS, JS, imagenes (UD5)
|   |       |-- templates/       <- plantillas HTML (UD5)
|   |       `-- application.properties
|   `-- test/java/es/decroly/tienda/   <- pruebas
|-- mvnw  /  mvnw.cmd            <- lanzadores de Maven
`-- pom.xml                      <- configuracion de Maven
```

Carpeta o fichero	Contenido
`src/main/java`	Tu código Java, organizado en paquetes.
`src/main/resources`	Todo lo que no es Java: configuración, plantillas HTML, CSS, imágenes.
`src/test/java`	Las pruebas automáticas. Maven las ejecuta antes de empaquetar.

Carpeta o fichero	Contenido
`target/`	Aparece al compilar: clases compiladas y el `.jar` final. Se genera sola; nunca se edita ni se sube a Git.
`mvnw`, `mvnw.cmd`, `.mvn/`	El **Maven Wrapper**: permite usar Maven sin instalarlo. La primera vez descarga la versión de Maven que necesita el proyecto.
`pom.xml`	La configuración del proyecto para Maven.

El fichero pom.xml, por partes

pom.xml (simplificado)

```
<!-- 1. PARENT: hereda la configuracion de Spring Boot -->
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.x.x</version>
</parent>

<!-- 2. COORDENADAS: identifican a este proyecto -->
<groupId>es.decroly</groupId>
<artifactId>tienda-decroly</artifactId>
<version>0.0.1-SNAPSHOT</version>

<!-- 3. PROPIEDADES -->
<properties>
    <java.version>21</java.version>
</properties>

<!-- 4. DEPENDENCIAS: librerias que usa el proyecto -->
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<!-- 5. BUILD: plugin que genera el jar ejecutable -->
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
```

```
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

Parte	Qué hace
1. parent	Hereda la configuración de Spring Boot, que incluye la lista de versiones compatibles de todas las librerías. Por eso las dependencias no llevan `<version>`.
2. Coordenadas	`groupId` (organización), `artifactId` (nombre del proyecto) y `version` identifican a cualquier librería o proyecto Maven. *SNAPSHOT* indica que es una versión en desarrollo.
3. properties	Valores de configuración; aquí, la versión de Java.
4. dependencies	Cada `<dependency>` se identifica por sus coordenadas. El `scope` indica cuándo se usa: `runtime` solo al ejecutar, `test` solo en las pruebas.
5. build	Plugins que intervienen al construir. `spring-boot-maven-plugin` empaqueta el jar ejecutable y permite `mvnw spring-boot:run`.

Los starters

Un starter no es una librería, sino un **paquete de librerías**. `spring-boot-starter-web` trae, entre otras, Spring MVC, el servidor Tomcat y Jackson (la librería que convierte objetos Java a JSON). En IntelliJ puedes verlas todas en el panel **Maven › Dependencies**.

En Spring Boot 4 puede que el asistente escriba `spring-boot-starter-webmvc` en lugar de `spring-boot-starter-web`. Son equivalentes para lo que haremos.

El ciclo de vida de Maven

Maven construye un proyecto siguiendo siempre las mismas **fases**, en orden. Cuando pides una fase, Maven ejecuta también todas las anteriores:

Fase	Qué hace
`clean`	Borra la carpeta `target` (empezar de cero). Es independiente de las demás.
`compile`	Compila el código de `src/main/java`.
`test`	Compila y ejecuta las pruebas de `src/test/java`. Si alguna falla, se detiene.

Fase	Qué hace
`package`	Empaqueta el proyecto en `target/tienda-decroly-0.0.1-SNAPSHOT.jar`.
`install`	Copia el jar a tu repositorio local `.m2` para que otros proyectos lo usen.

Los paquetes: cómo se organiza el código

El paquete que indicas al crear el proyecto (`es.decroly.tienda`) es el **paquete raíz**. Dentro crearás un subpaquete por capa. Así será tienda-decroly al final del bloque:

Paquetes de tienda-decroly

```
es.decroly.tienda
|-- TiendaDecrolyApplication.java
|-- controllers/    <- capa Controller
|-- services/       <- capa Service
|-- repositories/   <- capa Repository
|-- domain/         <- clases del negocio: Producto, Familia
|-- dto/            <- objetos para enviar y recibir datos
|-- config/         <- clases de configuración
`-- exceptions/     <- errores propios de la aplicación
```

Esta organización se llama **por capas**. Otra opción habitual en proyectos grandes es organizar **por funcionalidad** (un paquete `productos` con su controlador, servicio y repositorio; otro `pedidos`…). Nosotros usaremos la organización por capas porque hace más visible la arquitectura que estás aprendiendo.

La clase principal y @SpringBootApplication

TiendaDecrolyApplication.java

```java
package es.decroly.tienda;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class TiendaDecrolyApplication {

    public static void main(String[] args) {
        SpringApplication.run(TiendaDecrolyApplication.class, args);
    }
}
```

Es una clase Java con un `main` normal. La única línea del `main` pone en marcha Spring. La anotación `@SpringBootApplication` es, en realidad, la suma de tres anotaciones:

Anotación incluida	Qué hace
`@SpringBootConfiguration`	Indica que esta clase puede contener configuración de Spring.
`@EnableAutoConfiguration`	Activa la autoconfiguración: Spring Boot se configura según las librerías que encuentra.
`@ComponentScan`	Recorre **este paquete y todos sus subpaquetes** buscando clases anotadas (`@RestController`, `@Service`…) para crear sus beans.

> **ATENCIÓN · Regla de oro de los paquetes**
> Por culpa de `@ComponentScan`, todas tus clases deben estar dentro de `es.decroly.tienda` o de un subpaquete. Si creas una clase en otro paquete (por ejemplo, `es.decroly.controllers`), Spring no la encontrará y tu endpoint devolverá un error 404 aunque el código sea correcto.

El fichero application.properties

Está en `src/main/resources` y contiene la **configuración** de la aplicación en formato `clave=valor`. Como Spring Boot aplica valores por defecto, al principio casi está vacío. Aquí tienes algunos ejemplos de lo que se puede configurar:

application.properties

```
spring.application.name=tienda-decroly
# Puerto del servidor (por defecto, 8080)
server.port=8080
# Nivel de detalle del log de nuestro paquete
logging.level.es.decroly.tienda=DEBUG
```

Mas adelante aprenderás a crear tus propias propiedades y en la UD4 añadirás aquí los datos de conexión a MySQL.

Práctica 1. Crea el proyecto

Ya sabes cómo es un proyecto Spring Boot por dentro. Ahora vas a generarlo con **Spring Initializr**, el asistente oficial.

Prepara el entorno

1. Abre una terminal (en Windows, `cmd` o PowerShell) y comprueba tu versión de Java con `java -version`. Necesitas la **17 o superior**; te recomendamos una LTS (21 o 25).

2. Si no tienes JDK, IntelliJ puede descargarlo por ti al abrir el proyecto (**File › Project Structure › SDK › Download JDK**).
3. Abre IntelliJ IDEA. Sirve cualquier edición: la gratuita y la Ultimate funcionan igual para todo este tema.

Genera el proyecto

Entra en `https://start.spring.io` y rellena el formulario:

Campo	Valor	Relación con lo que has estudiado
Project	**Maven**	El gestor del proyecto.
Language	**Java**	
Spring Boot	La versión estable más alta	Será la versión del `parent`. Evita las que terminan en SNAPSHOT o M1, M2… (versiones de prueba).
Group	`es.decroly`	Coordenada `groupId`.
Artifact	`tienda-decroly`	Coordenada `artifactId`.
Name	`tienda-decroly`	De aquí sale el nombre de la clase principal: `TiendaDecrolyApplication`.
Description	Backend de la tienda online Decroly	Descripción libre.
Package name	`es.decroly.tienda`	El **paquete raíz**. Escríbelo a mano: el que propone el asistente no es válido por el guion.
Packaging	**Jar**	Jar ejecutable con Tomcat embebido.
Java	La versión de tu JDK	Propiedad `java.version`. Debe ser igual o menor que la de tu JDK.

Pulsa **ADD DEPENDENCIES** y añade estos dos starters:

- **Spring Web**: aplicaciones web y servicios REST, con Tomcat embebido.
- **Spring Boot DevTools**: reinicia la aplicación automáticamente cuando cambias el código.

1. Pulsa **GENERATE**. Se descargará `tienda-decroly.zip`.
2. Descomprímelo en tu carpeta de trabajo (por ejemplo, `C:\DWES\tienda-decroly`). Evita rutas con tildes o espacios.

3. En IntelliJ, elige **File › Open**, selecciona la carpeta `tienda-decroly` (la que contiene `pom.xml`) y pulsa **OK**. Si pregunta, elige **Trust Project**.
4. Espera a que Maven descargue las dependencias (barra de progreso abajo a la derecha). La primera vez tarda unos minutos: está llenando tu repositorio `.m2`.
5. Recorre el proyecto en el panel **Project** y localiza cada elemento de la estructura del apartado 3. Abre `pom.xml` y encuentra sus cinco partes.
6. Abre el panel **Maven** (a la derecha) y despliega **Dependencies**. Comprueba cuántas librerías ha traído el starter de Spring Web.

> **Con IntelliJ Ultimate**
> La edición Ultimate trae Spring Initializr integrado: **File › New › Project › Spring Boot**. Se rellenan los mismos campos y el resultado es idéntico.

4. Qué pasa al arrancar una aplicación Spring Boot

Cuando ejecutas la clase principal ocurren, en unos pocos segundos, estos pasos:

1. La máquina virtual de Java ejecuta el método `main`, como en cualquier programa.
2. `SpringApplication.run(...)` crea el **contenedor** de Spring.
3. Spring lee la configuración de `application.properties`.
4. **Escaneo de componentes:** recorre el paquete raíz y sus subpaquetes, encuentra las clases anotadas y crea sus beans.
5. **Autoconfiguración:** detecta las librerías del proyecto y configura lo necesario. Como tienes Spring Web: configura Spring MVC, Jackson y un servidor Tomcat.
6. Arranca **Tomcat**, que empieza a escuchar en el puerto 8080.
7. La aplicación queda **en espera** de peticiones. No termina, como haría un programa de consola: sigue viva hasta que la detienes.

Todo esto queda reflejado en el **log**, la salida de la consola. Estas son las líneas que conviene saber leer:

Línea del log (resumida)	Qué significa
`Starting TiendaDecrolyApplication using Java 21…`	Empieza el arranque; indica la versión de Java.
`No active profile set, falling back to "default"`	Usa la configuración por defecto (los perfiles se ven más adelante).
`Tomcat initialized with port 8080 (http)`	La autoconfiguración ha preparado Tomcat.

Línea del log (resumida)	Qué significa
`Tomcat started on port 8080 (http) with context path '/'`	El servidor ya acepta peticiones.
`Started TiendaDecrolyApplication in 2.3 seconds`	Arranque completado.

Cada línea del log sigue el mismo formato: fecha y hora, **nivel** (`INFO`, `WARN`, `ERROR`), número de proceso, **hilo** entre corchetes (`[main]`), **clase** que escribe el mensaje y el mensaje. Cuando algo falla, busca la primera línea con nivel `ERROR` y lee el mensaje que empieza por `Caused by:`.

Práctica 2. Arranca y empaqueta la aplicación

1. Abre `TiendaDecrolyApplication.java` y pulsa el triángulo verde junto a `public class`. Elige **Run**.
2. Localiza en el log las líneas de la tabla anterior.
3. Abre el navegador en `http://localhost:8080`. Verás una **Whitelabel Error Page** con un error 404: el servidor funciona, pero todavía no hay ningún controlador que atienda esa dirección.
4. Detén la aplicación con el cuadrado rojo (**Stop**). Recarga el navegador: ahora el error es distinto, porque ya no hay nadie escuchando en el puerto 8080. ¿Qué mensaje da el navegador?
5. Cambia el puerto añadiendo `server.port=8081` en `application.properties`, arranca de nuevo y comprueba en el log y en el navegador que ha cambiado. Después, deja el 8080.

Maven desde la terminal

IntelliJ usa Maven por debajo, pero también puedes usarlo tú desde la terminal integrada (**View › Tool Windows › Terminal**):

Terminal

```
# Arrancar la aplicacion (Windows / Linux-macOS)
mvnw.cmd spring-boot:run
./mvnw spring-boot:run

# Borrar target, compilar, probar y empaquetar
mvnw.cmd clean package

# Ejecutar el paquete generado
java -jar target/tienda-decroly-0.0.1-SNAPSHOT.jar
```

Ejecuta `clean package` y observa en la salida cómo Maven recorre las fases del ciclo de vida. Después ejecuta el jar: ese único fichero contiene tu código, todas las librerías y Tomcat. Se puede copiar a cualquier máquina con Java y ejecutarlo tal cual: así se **despliega** una aplicación Spring Boot.

5. Cómo llega una petición HTTP a tu código

Repaso: petición y respuesta HTTP

Toda la comunicación web sigue el modelo **petición-respuesta**: el cliente envía una petición y el servidor devuelve una respuesta. Cuando escribes `http://localhost:8080/saludo` en el navegador, se envía un mensaje de texto como este:

Petición HTTP

```
GET /saludo HTTP/1.1          <- metodo, ruta y version
Host: localhost:8080          <- cabeceras
Accept: text/html
                              <- linea en blanco (GET sin cuerpo)
```

Respuesta HTTP

```
HTTP/1.1 200                        <- codigo de estado
Content-Type: text/plain;charset=UTF-8
Content-Length: 34
                                    <- linea en blanco
Hola Mundo desde la Tienda Decroly  <- cuerpo
```

Puedes ver estos mensajes reales en el navegador: pulsa F12, pestaña **Red** (Network), recarga la página y haz clic en la petición.

Elemento	Para qué sirve	Ejemplo
Método	Qué quiere hacer el cliente	`GET` (leer), `POST` (crear), `PUT` (modificar), `DELETE` (borrar)
Ruta	Sobre qué recurso actúa	`/saludo`, `/api/productos/7`
Cabeceras	Información adicional	`Content-Type` indica el formato del cuerpo
Cuerpo	Los datos que se envían	Un texto, un JSON, un HTML
Código de estado	Resultado de la operación	`200` correcto, `404` no encontrado, `500` error del servidor

El recorrido dentro de Spring Boot

Dentro de la aplicación, la petición pasa por varias piezas antes de llegar a tu código:

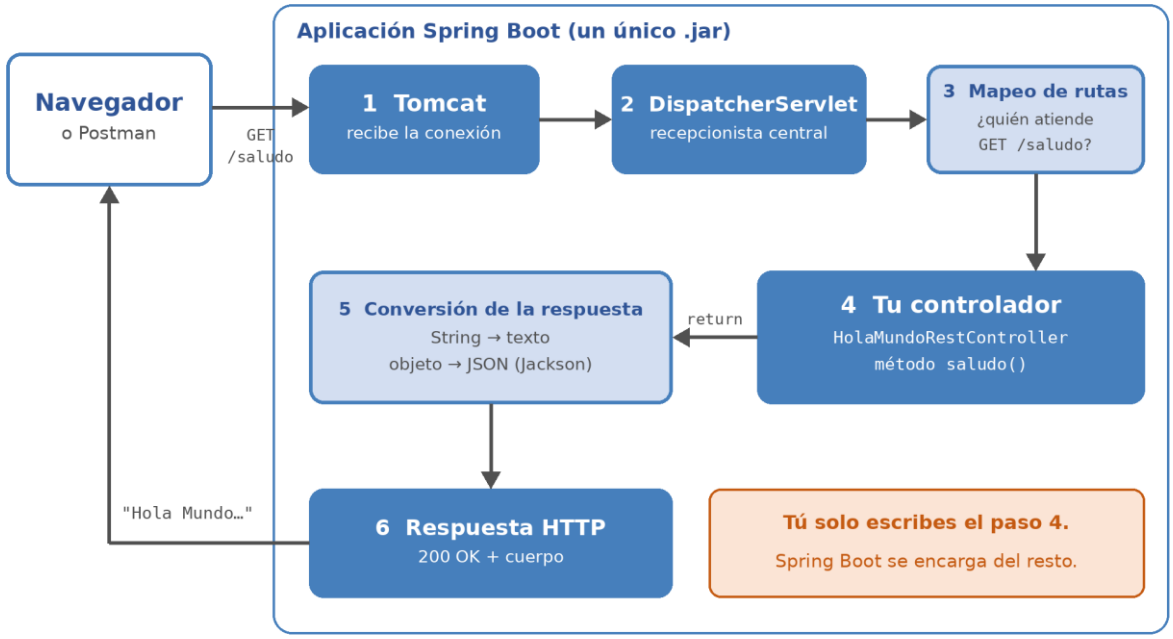


Figura 4. Recorrido de una petición GET /saludo dentro de una aplicación Spring Boot.

Paso	Quién	Qué hace
1	**Tomcat**	Recibe la conexión de red y traduce el texto HTTP a objetos Java.
2	**DispatcherServlet**	El "recepcionista" de Spring MVC. **Todas** las peticiones pasan por él, y él decide a quién enviárselas.
3	**Mapeo de rutas**	Al arrancar, Spring apuntó en una tabla qué método atiende cada combinación de método HTTP y ruta. El DispatcherServlet la consulta.
4	**Tu controlador**	Se ejecuta el método que corresponde. **Es lo único que escribes tú.**
5	**Conversión**	Lo que devuelve tu método se convierte al formato de la respuesta: un `String` se envía como texto; un objeto se convierte a JSON con Jackson.
6	**Respuesta**	Tomcat envía la respuesta al cliente con su código de estado.

El patrón Front Controller

Que todas las peticiones entren por un único punto (el DispatcherServlet) es un **patrón de diseño** llamado *Front Controller*. Tener una única puerta de entrada permite aplicar en un solo sitio cosas comunes a todas las peticiones, como la seguridad o el tratamiento de errores.

6. Qué es un controlador

Un **controlador** es una clase de la capa Controller: la puerta de entrada de la aplicación. Es la clase que recibe las peticiones que el DispatcherServlet le reparte. Un controlador tiene cuatro responsabilidades:

1. **Recibir** la petición que le corresponde (según su ruta y su método HTTP).
2. **Extraer los datos** que trae: de la ruta, de los parámetros, del cuerpo.
3. **Delegar** el trabajo en la capa de servicios.
4. **Construir la respuesta**: los datos y el código de estado.

Tan importante como lo que hace es lo que **no** debe hacer: calcular precios, validar reglas del negocio o acceder a la base de datos. Un buen controlador es **corto**: sus métodos suelen tener pocas líneas.

Endpoint, recurso y API

Concepto	Definición	Ejemplo en la tienda
Recurso	Cualquier cosa del negocio que se pueda consultar o manipular.	Un producto, una familia, un pedido.
Endpoint	Combinación de método HTTP y ruta que atiende un método de un controlador.	`GET /api/productos/7` → obtener el producto 7.
API	El conjunto de todos los endpoints de la aplicación: su "interfaz" con el mundo exterior.	La API de tienda-decroly.

@Controller y @RestController

Spring tiene dos anotaciones para declarar controladores. La diferencia está en **qué hace Spring con lo que devuelve el método**:

	`@Controller`	`@RestController`
El método devuelve…	El **nombre de una vista** (una plantilla HTML)	Los **datos** que forman el cuerpo de la respuesta
El cliente recibe…	Una página HTML generada en el servidor	Texto o JSON
Se usa para…	Aplicaciones web con páginas (UD5)	APIs REST (UD3 y UD4)

En realidad, `@RestController` es la suma de `@Controller` y `@ResponseBody`. Esta última anotación le dice a Spring: "lo que devuelve este método no es el nombre de una vista, escríbelo directamente en el cuerpo de la respuesta".

Cómo se asocia una ruta a un método

Las anotaciones de mapeo indican qué endpoint atiende cada método. Hay una por cada método HTTP:

Anotación	Atiende	Uso típico (lo verás en 3.2)
`@GetMapping("/ruta")`	GET	Consultar
`@PostMapping("/ruta")`	POST	Crear
`@PutMapping("/ruta")`	PUT	Modificar
`@DeleteMapping("/ruta")`	DELETE	Borrar
`@RequestMapping("/ruta")`	Cualquiera	Sobre la clase: prefijo común para todas sus rutas

Fíjate en que la dirección pública la decide la **anotación**, no el nombre del método Java. El método podría llamarse de cualquier forma.

Práctica 3. Tu primer controlador

Hola Mundo REST

1. Haz clic derecho sobre el paquete `es.decroly.tienda` › **New › Package** y escribe `controllers`.
2. Clic derecho sobre `controllers` › **New › Java Class** (o Alt+Insert) y llámala `HolaMundoRestController`. Por convenio, los controladores REST terminan en `RestController`.
3. Escribe el siguiente código:

HolaMundoRestController.java

```
package es.decroly.tienda.controllers;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HolaMundoRestController {

    @GetMapping("/saludo")
    public String saludo() {
        return "Hola Mundo desde la Tienda Decroly";
    }
}
```

Línea	Qué hace y con qué parte de la teoría se relaciona
`package es.decroly.tienda.controllers;`	Subpaquete del paquete raíz: `@ComponentScan` lo encontrará.
`@RestController`	Al arrancar, Spring crea un bean de esta clase y registra sus rutas. Lo que devuelvan sus métodos irá al cuerpo de la respuesta.
`@GetMapping("/saludo")`	Añade a la tabla de mapeo la entrada `GET /saludo` → `saludo()`
`return "Hola…";`	El `String` se convierte en el cuerpo de la respuesta con `Content-Type: text/plain`

1. Reinicia la aplicación y entra en `http://localhost:8080/saludo`.
2. Abre las herramientas del navegador (F12 › **Red**), recarga y comprueba el código de estado y la cabecera `Content-Type` de la respuesta.
3. Cambia la ruta a `@GetMapping("/hola")`, reinicia y prueba las dos direcciones. ¿Qué código de estado devuelve ahora `/saludo`? ¿Por qué?

Recibir datos en la petición

Un endpoint que siempre responde lo mismo no es muy útil. En una petición GET, el cliente puede enviar datos de dos formas:

	Variable de ruta: `@PathVariable`	Parámetro de consulta: `@RequestParam`
Aspecto de la URL	`/productos/7`	`/productos?familia=perifericos`

	Variable de ruta: `@PathVariable`	Parámetro de consulta: `@RequestParam`
El dato forma parte…	De la propia ruta	De la cadena que va detrás de `?`
Uso típico	Identificar **un recurso** concreto	Filtrar, buscar, ordenar o paginar
¿Obligatorio?	Sí: sin él, la ruta es otra	Opcional si se indica `defaultValue`

Añade estos dos métodos a `HolaMundoRestController`. IntelliJ te propondrá importar `PathVariable` y `RequestParam`: pulsa Alt+Enter.

HolaMundoRestController.java (añade estos métodos)

```
// Variable de ruta:  http://localhost:8080/hola/Ana
@GetMapping("/hola/{nombre}")
public String saludoPersonalizado(@PathVariable String nombre) {
    return "Hola " + nombre + ", bienvenido/a a la Tienda Decroly";
}

// Parametro de consulta:  http://localhost:8080/buscar?texto=raton
@GetMapping("/buscar")
public String buscar(@RequestParam(defaultValue = "todo") String texto) {
    return "Buscando productos que contengan: " + texto;
}
```

En el primer método, `{nombre}` es un hueco de la ruta, y `@PathVariable` copia su valor en el parámetro Java del mismo nombre. En el segundo, `@RequestParam` busca en la URL un parámetro llamado `texto`; si no viene, usa `"todo"`. Spring **convierte** automáticamente el texto de la URL al tipo del parámetro Java (`int`, `double`…). Si no puede, responde con un error **400 Bad Request**.

Experimenta

- Prueba `/buscar` sin parámetro y con `?texto=teclado`.
- Quita el `defaultValue` y vuelve a probar `/buscar` sin parámetro. ¿Qué código de estado obtienes?
- Prueba `/hola` sin nombre. ¿Por qué da 404 y no 400?

7. Herramientas de desarrollo

DevTools: reinicio rápido

Cada cambio en el código Java obliga a reiniciar la aplicación, porque las clases ya están cargadas en memoria. **DevTools** acelera este proceso: separa las librerías (que no cambian) de tus clases, y al detectar una recompilación solo vuelve a cargar tus clases. El reinicio pasa de varios segundos a menos de uno. DevTools solo actúa durante el desarrollo; se desactiva automáticamente al ejecutar el jar empaquetado.

Depurar una aplicación de servidor

Depurar un programa de consola es sencillo: lo ejecutas y sigue su camino desde el `main`. Una aplicación de servidor es distinta:

- El programa **no termina**: arranca y se queda esperando.
- Tu código **no se ejecuta cuando tú quieres**, sino cuando llega una petición.
- Cada petición se atiende en un **hilo** propio. En el log lo verás como `[nio-8080-exec-1]`, `[nio-8080-exec-2]`…

Por eso la técnica es: arrancar en modo depuración, poner un **punto de ruptura** en la línea que interesa y **provocar la petición** desde el navegador. Cuando la petición llega a esa línea, la ejecución se congela y puedes examinarla.

Acción	Atajo	Qué hace
Punto de ruptura	Clic en el margen	Marca una línea donde se detendrá la ejecución.
Debug	Mayús+F9	Arranca la aplicación en modo depuración.
Step Over	F8	Ejecuta la línea actual y pasa a la siguiente.
Step Into	F7	Entra dentro del método que se llama en la línea actual.
Evaluate Expression	Alt+F8	Calcula el valor de cualquier expresión con los datos del momento.
Resume	F9	Continúa la ejecución hasta el siguiente punto de ruptura.

Logger en lugar de System.out.println

Es común que en muchos ejemplos podáis ver el uso de `System.out.println` para ver por dónde pasa la ejecución. En una aplicación real se usa un **logger**, porque cada mensaje lleva fecha, hora, hilo, clase de origen y **nivel**, y en producción se puede enviar a un fichero o filtrar sin tocar el código:

Nivel	Cuándo se usa
`log.error(…)`	Algo ha fallado y hay que revisarlo.
`log.warn(…)`	Situación anómala que no impide continuar.
`log.info(…)`	Hechos importantes del funcionamiento normal.
`log.debug(…)`	Detalles para el desarrollador. No se muestran salvo que se active ese nivel en `application.properties`.

Práctica 4. Recarga automática con DevTools

1. Con la aplicación en marcha, cambia el texto que devuelve `saludo()`.
2. Pulsa **Ctrl+F9** (*Build Project*). En la consola verás que la aplicación se reinicia sola en menos de un segundo.
3. Recarga el navegador: ya aparece el texto nuevo.

> **Recarga totalmente automática (opcional)**
> Si no quieres ni pulsar Ctrl+F9, activa en **Settings › Build, Execution, Deployment › Compiler** la opción **Build project automatically** y, en **Settings › Advanced Settings**, la opción **Allow auto-make to start even if developed application is currently running**.

Práctica 5. Encuentra el error con el depurador

Crea en `controllers` la clase `PrecioRestController`. Compila y se ejecuta sin errores, pero **calcula mal**:

PrecioRestController.java

```
package es.decroly.tienda.controllers;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class PrecioRestController {

    private static final Logger log =
            LoggerFactory.getLogger(PrecioRestController.class);

    // Ejemplo: http://localhost:8080/precio/100?iva=21
    @GetMapping("/precio/{base}")
    public String calcularPrecio(
            @PathVariable double base,
            @RequestParam(defaultValue = "21") int iva) {
        log.info("Calculando precio: base={} iva={}", base, iva);
        double total = base * (1 + iva / 100);
        return "Sin IVA: " + base + " EUR. Con IVA: " + total + " EUR";
    }
}
```

En `log.info`, cada `{}` se sustituye por los argumentos siguientes, en orden. Prueba `http://localhost:8080/precio/100?iva=21`: debería dar 121, pero da 100. Averigua por qué:

1. Detén la aplicación. Haz clic en el margen izquierdo de la línea `double total = …` para poner un **punto de ruptura**.
2. Arranca en modo depuración: triángulo verde › **Debug** (Mayús+F9).
3. Vuelve a cargar la URL. El navegador se queda esperando: la petición está congelada en tu línea. Fíjate en el nombre del hilo en el panel **Debugger**.
4. En el panel **Variables**, comprueba los valores de `base` e `iva`. ¿Son los que esperabas?
5. Selecciona la expresión `iva / 100`, clic derecho › **Evaluate Expression** (Alt+F8), y observa el resultado.
6. Pulsa **Step Over** (F8) y observa el valor de `total`.
7. Explica la causa del error, corrígelo, pulsa **Resume** (F9) y comprueba el resultado.

Errores frecuentes

Síntoma	Causa probable	Solución
`Port 8080 was already in use`	Ya hay otra instancia de la aplicación arrancada.	Detén todas las instancias en la pestaña **Run** o **Services**, o cambia `server.port`.
Whitelabel Error Page 404 en una ruta que sí existe	La clase está fuera del paquete raíz o le falta `@RestController`.	Revisa el `package` y las anotaciones. Reinicia.
Error 400 Bad Request	Falta un parámetro obligatorio o no se puede convertir (por ejemplo, `/precio/abc`).	Envía el parámetro con un valor del tipo correcto.
`release version XX not supported`	El JDK es más antiguo que la `java.version` del `pom.xml`.	Iguala el JDK en **Project Structure** con `java.version`.
Los `import` salen en rojo	Maven no ha terminado o falló la descarga.	Panel **Maven** › **Reload All Maven Projects**.
El cambio no se refleja	No se ha reiniciado ni recompilado.	Rerun (Ctrl+F5) o, con DevTools, Ctrl+F9.